■ MedConnect Healthcare System

Full Technical Architecture, System Workflows & Interview Preparation Guide

1. System Overview & Quick Access Credentials

MedConnect is a full-stack, enterprise-grade healthcare appointment and electronic medical record management system built using ASP.NET Core (.NET 8), Entity Framework Core Code-First, Microsoft SQL Server, and responsive MVC Razor Views. The system has been configured with pre-seeded dummy accounts and 1-Click Quick Demo Login buttons on the sign-in screen.

Role	Login Email	Password	Default Portal Route	Key Permissions
Admin	admin@medconnect.com	Admin@123	/Admin	Analytics, Doctor & Patient CRUD, Departments, Master Schedu
Doctor 1	dr.smith@medconnect.com	Doctor@123	/Doctor	Cardiology schedule, Slot Generator, Accept/Reject, EHR & Pre
Doctor 2	dr.sarah@medconnect.com	Doctor@123	/Doctor	Neurology schedule, Slot Generator, Complete Consultations, M
Patient 1	john.doe@example.com	Patient@123	/Patient	Search Specialists, Interactive Slot Picker, Track Bookings, Viev
Patient 2	emily.watson@example.com	Patient@123	/Patient	Schedule appointments, Cancel visits, Profile management

2. Core System Workflows & User Journeys

A. Patient Consultation & Booking Workflow

Step 1 - Doctor Discovery: Patient searches doctors by clinical specialty, department, fee, or credentials.
Step 2 - Slot Selection: Patient selects a consultation date. An AJAX call dynamically retrieves open slots for that doctor on that date.
Step 3 - Conflict-Free Booking: The system validates that (a) date is not in the past, (b) doctor is not already booked at that slot, and (c) patient has no overlapping appointments.
Step 4 - Appointment Created: The appointment is persisted in Pending status, and the doctor's availability slot is marked IsBooked = true.
Step 5 - Follow-up & Records: After the doctor completes the consultation, the patient accesses the digital prescription (`Print Rx`) and medical findings in /Patient/MyMedicalRecords.

B. Doctor Schedule & Clinical EHR Workflow

Step 1 - Availability Setup: Doctor uses the slot generator modal to specify date, start time, end time, and duration (e.g. 30 mins). Non-overlapping slots are persisted.
Step 2 - Review Schedule: Doctor views pending patient appointments in tabbed views (Today, Upcoming, Completed).
Step 3 - Accept / Reject: Doctor clicks 'Accept' to confirm, or 'Reject' to decline (which immediately frees up the availability slot).
Step 4 - Clinical Entry (EHR): Doctor conducts visit and clicks 'Complete & Prescribe' to input primary diagnosis, observed symptoms, medications with dosages, and clinical notes.
Step 5 - Auto-Completion: The appointment status automatically transitions to Completed and locks against further cancellations.

C. Administrative Operations Workflow

Real-Time KPI Monitoring: Executive dashboard presents total revenue, active doctors, registered patients, and appointment status distribution.
Clinician Onboarding: Full CRUD interface to add doctors, assign department specialties, configure fee rates, and toggle

account availability.
Master Control: Search and filter all hospital bookings with status override capability and departmental utilization analytics.

3. Technical Architecture & Design Patterns

The application is structured into four decoupled layers following **Clean Architecture** principles:

Layer	Project	Responsibilities
Domain	my_demo_application.Domain	Pure POCO entities (User, Doctor, Patient, Department, DoctorAvailability, Appointment, Medica
Application	my_demo_application.Application	Business logic contracts (IApplicationDbContext, IAuthService, IAdminService, IDoctorService, I
Infrastructure	my_demo_application.Infrastructure	EF Core ApplicationDbContext, SQL Server configuration, Migrations, Seed Data Initializer, Gen
Presentation	my_demo_application (Web)	ASP.NET Core MVC Controllers, Razor Views with Bootstrap 5 + FontAwesome 6, RESTful API

4. Relational Database Schema (SQL Server)

The database **MedConnectDb** is hosted on SQL Server (Aditya\SQLEXPRESS). Key schema relationships include:
• **Users •—• Doctors / Patients:** 1-to-1 relationship with unique foreign key and cascade deletion.
• **Departments •—< Doctors:** 1-to-many relationship with Restrict delete behavior (preventing orphaned doctor assignments).
• **Doctors •—< DoctorAvailabilities:** 1-to-many relationship with composite index on (DoctorId, AvailableDate).
• **Appointments •—• MedicalRecords:** 1-to-1 relationship linking clinical notes to the appointment.
• **Appointments >—• Doctors / Patients:** Foreign key constraints with Restrict behavior protecting transaction integrity.

5. Top Technical Interview Questions & Answers

Study these comprehensive answers to excel during technical screenings and full-stack interviews:

Q1: Can you give an architectural overview of MedConnect?

MedConnect is a full-stack healthcare appointment and medical management system built with ASP.NET Core (.NET 8), Entity Framework Core Code-First, and SQL Server. I designed it using Clean Architecture with four decoupled layers: Domain (pure entities and enums), Application (business logic services, interfaces, DTOs), Infrastructure (EF Core ApplicationDbContext, repositories, migrations), and Presentation (MVC Razor views and RESTful API controllers with Swagger). It features dual authentication—cookie authentication for responsive web views and JWT Bearer for APIs—with role-based access for Admin, Doctor, and Patient.

Q2: Why did you choose Clean Architecture instead of putting everything in a single project?

Clean Architecture enforces the Dependency Inversion Principle. The core business rules and domain entities remain independent of UI frameworks and database libraries. In MedConnect, the Application layer depends only on Domain interfaces (like IApplicationDbContext), meaning we could swap SQL Server for PostgreSQL or replace Razor views with React without altering a single line of business logic. It also makes unit testing effortless via mocking.

Q3: How did you implement dual authentication (Cookies + JWT) in ASP.NET Core?

In Program.cs, I registered both CookieAuthenticationDefaults and JwtBearerDefaults. Cookies serve as the default scheme for web users with sliding expiration, while JWT Bearer is configured for API controllers. The API endpoints specify [Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme, Roles = "...")]. This gives the application the versatility to power both server-rendered web portals and mobile/SPA clients.

Q4: How does MedConnect prevent race conditions and double-booking of appointments?

Prevention happens at two levels: First, in the business logic (PatientService.BookAppointmentAsync), we execute queries to verify that neither the doctor nor the patient has another active appointment at the requested date and start time. Second, in the database schema, we maintain indexed fields on (DoctorId, AvailableDate) and update the slot's IsBooked flag inside the transaction. If a conflict occurs, a descriptive error is returned.

Q5: How did you handle password security?

We never store plaintext passwords. We utilize BCrypt.Net-Next with salted hashing. During registration, BCrypt.HashPassword(password) hashes the password before persistence. During authentication, BCrypt.Verify(password, user.PasswordHash) compares the credentials in constant time to prevent timing attacks. Password hashes are excluded from all ViewModels and API responses.

Q6: What is the difference between IEnumerable, IQueryable, and List in your LINQ queries?

IQueryable executes query filters directly inside SQL Server via T-SQL translation (deferred execution), which is critical for performance when filtering large datasets like appointments. IEnumerable operates in-memory on data already loaded. In MedConnect, we apply filters and search clauses on IQueryable before calling .ToListAsync(), minimizing memory consumption and network overhead.

Q7: Explain the difference between AddScoped, AddTransient, and AddSingleton in ASP.NET Core.

AddScoped creates an instance once per HTTP request lifecycle, which is ideal for DbContext and business services (like IDoctorService). AddTransient creates a new instance every time it is requested, best for lightweight stateless utilities. AddSingleton creates a single instance for the application lifetime, suitable for caching or configuration providers. In MedConnect, all services and DbContext use Scoped lifetime.

Q8: How did you prevent state transition errors in appointments?

In DoctorService and AdminService, we implemented strict state guards. For example, an appointment marked 'Cancelled' cannot be transitioned to 'Completed', and an appointment marked 'Completed' cannot be cancelled. Furthermore, completing an appointment requires authoring a linked MedicalRecord with a valid diagnosis and prescription.

Q9: How did you handle EF Core Code-First migrations and seeding?

We configured entity relationships via Fluent API in OnModelCreating. We ran 'dotnet ef migrations add InitialCreate' and 'dotnet ef database update' to generate the schema on SQL Server. Additionally, we implemented a DbInitializer class that automatically verifies migrations and seeds dummy data on application startup, ensuring zero manual setup for developers cloning the repo.

Q10: How does your application handle global errors?

In production, app.UseExceptionHandler("/Home/Error") captures unhandled exceptions and displays a user-friendly error view while logging stack traces to the configured logging provider. In API controllers, all responses follow a standard ApiResponse envelope returning appropriate HTTP status codes (200 OK, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).

Q11: What design patterns did you implement in this project?

We implemented several enterprise patterns: (1) Repository Pattern and Unit of Work for decoupled data access, (2) Dependency Injection / Inversion of Control, (3) DTO / ViewModel Pattern to prevent over-posting vulnerabilities, and (4) Service Layer Pattern to encapsulate complex domain rules away from controllers.

Q12: If you were to scale MedConnect for 1 million patients, what would you add?

In version 2.0, I would implement: (1) Distributed Redis caching for doctor availability queries, (2) RabbitMQ / Azure Service Bus for asynchronous appointment confirmation emails and SMS alerts, (3) SignalR for live doctor-patient notifications, (4) Stripe / Razorpay payment gateway integration, and (5) Docker containerization with Kubernetes orchestration.